

GroupTrack V2.5 -- Complete Decision Log

Extracted from May 17, 2026 full-day session

GREEN = Decision made | ORANGE = Open issue requiring resolution

1. ANR Fixes (Completed)

1.1 WorkManager ANR

- MeshLogCleanupWorker: Changed ExistingPeriodicWorkPolicy.UPDATE to KEEP -- prevents re-trigger on every cold start
- Added 30-second initial delay to PeriodicWorkRequest -- first run doesn't compete with startup
- Added WorkManager.pruneWork() before scheduleMeshLogCleanup() -- clears stale finished jobs
- File: MeshUtilApplication.kt -- committed and deployed

1.2 MediaStore ANR (Root Cause)

- scanImportDirectory() contained legacy MediaStore.Downloads query for .convoy files
 - This query blocked main thread via ContentResolver.query() Binder IPC when Media Provider was busy indexing tile files
 - Confirmed via bugreport thread dump: main thread stuck in ContentProviderProxy.query() inside ConvoyViewModel init
 - Fix: Removed the MediaStore.Downloads query entirely -- ConvoyFileReceiver handles .convoy imports via intents now
 - scanImportDirectory() retained for processing files in convoy_import/ directory
 - Dispatchers.IO added to scanImportDirectory launch as belt-and-suspenders
 - File: ConvoyViewModel.kt -- committed and deployed
1. NO MediaStore queries anywhere in the app. This is a standing rule.

1.3 P10_T Device Findings

- P10_T advertises 4GB RAM but kernel sees only 2.9GB (Unisoc/Spreadtrum chipset reserves ~1.1GB)
- P10_T is NOT incompatible hardware -- same build loads in 1 second after MediaStore fix
- Tester had ~60GB of tiles in Downloads, causing Media Provider to run at 58% CPU
- Google Media Provider indexing large tile directories makes ALL ContentResolver queries slow via Binder IPC
- Factory reset fixes were misleading -- it was the MediaStore query + busy Media Provider, not accumulated state
- ServiceKeepAliveWorker lint warning: missing connectedDevice foregroundServiceType in AndroidManifest.xml -- needs fix

2. Device Assignments

- Android 1 (P50, 8624SBCEDF00001789): Release APK only. Acceptance test device.
 - Android 2 (EP103B, 24039703201775): Play Store release for field testing. V3.0 debug deferred until V2.5 complete.
 - P10_T (P10V07162601000362): Play Store release. ANR fixed. 3GB usable RAM but functional.
 - Debug APK never on Android 1. Release APK never on Android 2. Play Store only on P10_T.
 - Play Store and local APK use different signing keys -- must uninstall one before installing the other.
2. Never mix debug/release on a device. Leaves stale signing artifacts.

3. V2.5 Architecture Decisions

3.1 Development Approach

- Phase 0 scaffolding ships first -- ALL UI placeholders before any feature code
- 4-week timeline, all 7 tasks ship together
- Scaffolding-first: build the skeleton, hang features on it. No refactoring.
- Design all UI and rules before any coding. HTML panel mockups as reference.
- Each feature: PLAN, RESEARCH (xref), AGREE, BUILD
- One comprehensive patch per agreed design, not piecemeal changes

3.2 Two Separate Queues

- Tile queue (Task 2): downloads, refreshes, removes map tiles. Never uploads.
 - Spatial data queue (Task 7): collects uploads via share prompt. Separate from tile queue.
3. Two queues never mixed. Different job types, different managers, different priorities.

3.3 Two Data Sources

- LOCAL: default, always shown, free. All local spatial data.
- SERVER: enhanced, paygated (\$3/month). Browse shared data from AWS.
- Source selector toggle between Local/Server -- never mixed on same display.
- V2.5: Server is paygated STUB only. No server queries.
- V2.6: Server functional behind paygate.

3.4 Upload Rules

- Uploads are PER-ITEM, triggered by share prompt on save.
 - Uploads are NEVER area-based. No bulk upload.
 - Uploads are spatial data only -- no tiles EVER uploaded to AWS.
 - Server-side areas are DERIVED from accumulated shared items, not uploaded as entities.
 - V2.5: upload queue COLLECTS only. No processing, no network transmission.
 - V2.6: processing activated, accumulated items start uploading.
4. Never build V2.6 processing code during V2.5. Collection and UI only.

3.5 Download Rules

- Downloads use area for DISCOVERY -- find what's available on server.
- User selects SPECIFIC items (one or many tracks/routes) to download.
- Download is AREA-INCLUSIVE -- selected item + associated waypoints within coverage.
- An area on the server is a COLLECTION of tracks, routes, and waypoints (one or many).
- Downloaded spatial data AUTO-GENERATES tile download requests from item geometry.
- Downloads are STUB only in V2.5. Functional in V2.6.

3.6 Tile Rules

- Tiles are NEVER downloaded in isolation by the user.
 - Tiles ALWAYS follow spatial data automatically.
 - HIGH priority tiles when explicit track/route downloaded.
 - LOW priority tiles when area-based spatial data downloaded.
 - Tile refreshes NEVER standalone -- only as byproduct of spatial data download.
 - Tile coverage maps to route + track + waypoint GEOMETRY within area, NOT full bbox.
 - Manual area draw (existing) stays for initial tile downloads.
 - Two methods for initial tile download: manual area draw (broad) or spatial data request (targeted along geometry).
 - Remove area tiles is the ONLY explicit user tile action (storage management).
 - Tiles NEVER transit AWS. Always sourced from tile providers (Esri, Google, etc.).
 - No consolidated queue management view.
 - Existing queue panel filtered by priority. Job type IS priority.
5. Tile refreshes are never downloaded by area. They happen when spatial data is downloaded.

3.7 Area Concept

- Area selector is a SHARED component across all spatial data tasks.
- Areas are independent of rides -- rides just create areas.
- Area used for: trail download boundaries, tile download targeting, spatial data discovery.
- Tile areas mapped to route/track/waypoint GEOMETRY to minimize downloaded data.
- Area delete removes definition only -- data and tiles stay unless explicitly removed.
- Download spatial data auto-creates tile download request for supporting tiles.

4. Task-Specific Decisions

4.1 Task 1: Source Picker

- One 'Change Map Source' button in Display Panel (ConvoyDisplayPanel.kt).
- Flow: button -> 'Which map type?' (SAT/TOPO/TOPO+) -> cascading accordion source list -> refresh toggle switch.
- Accordion groups 16 sources by type: SAT, HYB, TOPO, STREET, TERRAIN, OUTDOOR.
- Refresh is a TOGGLE SWITCH after source selection, not a separate dialog.
- Refresh toggle ON = creates SOURCE_REFRESH job in tile queue.
- Map Viewer only until cutover at end of V2.5.
- Select Cart HUD: new button on ConvoyButtonBar, replaces long-press for cart selection.
- Cart HUD picker: slide-up panel with radio listing, max 5-10 radios.
- After cart selection: NodeDetailHud displays (Set as Lead, Remove, Dismiss).
- Long-press permanently reclaimed from cart HUD selection for waypoints (Task 5).
- Button bar order: GROUP | MY CART | SELECT CART | HIDE | GEAR.
- Do map bar heading labels (SAT/TOPO/TOPO+) update to reflect assigned source name?

4.2 Task 2: Tile Queue

- Existing ConvoyDownloadQueue enhanced with new job types.
- New job types: SOURCE_REFRESH, SPATIAL_FOLLOW, REMOVE_AREA.
- Priority: explicit track/route = high, area spatial = low, source refresh = medium, removal = lowest.
- No 'download tiles' or 'refresh tiles' buttons -- tiles follow spatial data.
- Remove area tiles: explicit user action for storage management.

4.3 Task 3: Spatial Trails

- Trail source catalog BUNDLED as app asset with coverage boundaries per source.
- User defines area -> system recommends overlapping sources from catalog.
- User selects which sources to import from recommendations.
- App downloads full source from agency URL (UGRC etc.).
- On-device filtering: ONLY trails within selected area imported, NOT entire source.
- No AWS involvement for trail distribution.
- Dedup on ingestion: bbox pre-filter, length filter, point proximity, classification.
- Viewport lazy-load to map from local db -- pattern reused in Tasks 4, 5, 6.
- Replace static utah_trails_stgeorge.geojson asset with database queries.
- Preserve full trail geometry precision -- V3.0 snap-to depends on it.
- Trail type filter: lives in Display Panel or separate toggle panel?
- Source recommendation panel: inline in download flow or separate screen?
- Import progress indicator: toast or dedicated panel?

4.4 Task 4: Spatial Tracks

- Existing 'work with tracks' functions enhanced with area-based filtering.
- Track list controlled by map boundary.
- ALL GPX/KML imports are TRACKS. Always. No exceptions.
- Convert to route = type flip in shared table. Reversible.
- Track and route share SAME TABLE with type field (TRACK/ROUTE).
- No file sync on launch -- all imports user-initiated.
- Track source of truth remains GPX/KML file.
- Spatial db holds metadata + bbox as index, not file copy.
- Alias model: identity = file, display names in track_aliases.
- Download track auto-generates HIGH PRIORITY tile request for track geometry.
- FIT button: RESTORE lost feature -- zoom map to track bounds on selection.
- New track actions: DOWNLOAD, UPLOAD, TO ROUTE, DELETE, RENAME.
- LOCAL/SERVER toggle in track picker footer. Server paygated (V2.6 stub).
- Track FIT implementation: how does fitBounds work with WebView/JavaScript bridge?
- How does area filter integrate with existing track picker UI?

4.5 Task 5: Waypoints

- Long-press on map = waypoint creation.
- 10 types: trailhead, fuel, gate, hazard, scenic, water, camp, parking, rally, other.
- Proximity check: 100 meters (changed from original 25m).
- No move function -- delete and recreate.
- No consolidation -- delete the extra.
- Identity = coordinates, display names in waypoint_aliases.
- Share prompt on save: 'Share this data? Y/N' populates upload queue.
- Share prompt controlled by app_setting and ride privacy toggle.
- Private ride toggle suppresses share prompt entirely.
- Map legend generated from waypoint_types table.
- Long-press handler: requires JavaScript bridge to WebView -- implementation details TBD.
- Waypoint action menu (edit/delete/upload/type change): where does it appear -- overlay on map or slide-up panel?

4.6 Task 6: Routes

- All imports are tracks. Convert to route is explicit type flip. Reversible.
- Create by: draw on map (point-to-point straight lines V2.5), snap-to-trail, convert from track, import GPX/KML.
- Future: CV-assisted path smoothing between points using satellite imagery (post-V2.5).
- Routes NEVER independently shared -- only through ride enrollment.
- Route sharing controlled by configurable app_setting.
- Delete blocked if type = ROUTE and assigned to active ride with enrollments.
- source_track_id links converted tracks to routes.

- Snap-to-trail depends on Task 3 trail geometry precision.
- CV evolution path: V2.5 straight lines -> future CV adjusts to visible trail geometry.
- Point-to-point drawing handler: JavaScript bridge implementation details TBD.
- Snap-to-trail logic: how does proximity matching work against trail database?
- Route action menu: where does it appear?

4.7 Task 7: Area Data Transfer

- V2.5: upload queue infrastructure BUILT, share prompt COLLECTING, queue NOT PROCESSING.
- V2.6: upload processing ACTIVATED, download functions WIRED.
- Upload triggered by share prompt on save -- per-item, never area-based.
- No 'Upload Data' button. Uploads happen through share prompt only.
- Upload queue is a local database table collecting entries.
- Pending upload count shown as passive indicator, not action button.
- Downloads: area search for discovery -> browse collections -> select items -> download.
- Area on server = collection of tracks, routes, waypoints (one or many).
- Download includes selected items + associated waypoints within coverage.
- Downloaded spatial data auto-generates tile requests from geometry.
- Server browsing and downloads paygated.
- Separate queue from tile queue.
- Where does pending upload count indicator live? Badge on screen? Status line?
- Download tracks/routes by area: where do these stubs live in the UI?

5. Implementation Boundaries

5.1 V2.5 Builds (4 weeks)

- Phase 0: Complete UI scaffolding
- Tasks 1-6: Fully functional
- Task 7: Upload queue collects, download stubs only
- No AWS backend integration
- No network upload code
- Server browse UI is paygated stub

5.2 V2.6 Activates

- Upload queue processing -- items start uploading
- Download functions wired to AWS API
- Server browse UI functional behind paygate
- Ride enrollment download triggers

5.3 Hard Boundaries

6. V2.5 upload queue collects items locally. Does NOT send to AWS.
7. V2.5 share prompt works and populates queue table. No network calls.
8. V2.5 download menu sections exist as stubs. Not wired to AWS.
9. V2.5 server browse UI shows paygate prompt. No server queries.
10. Never build processing code during V2.5. Only collection and UI.

6. Open Issues Requiring Resolution

6.1 UI Placement

- Map bar heading labels: do they update to reflect assigned source name or stay fixed?
- Trail type filter: Display Panel toggle or separate panel?
- Source recommendation panel for trails: inline or separate screen?
- Import progress indicator: toast or panel?
- Pending upload count: badge, status line, or other?
- Download tracks/routes by area: location in UI?
- Waypoint action menu: overlay on map or slide-up panel?
- Route action menu: overlay on map or slide-up panel?

6.2 Technical Implementation

- Track FIT: how does fitBounds work with WebView/JavaScript bridge?
- Long-press waypoint: JavaScript bridge implementation between Kotlin and WebView
- Point-to-point route drawing: JavaScript bridge implementation
- Snap-to-trail: proximity matching algorithm against trail database
- ServiceKeepAliveWorker lint warning: missing foregroundServiceType in manifest

6.3 Data Model

- Track/route shared table: exact schema columns and type field values?
- Upload queue table: exact schema for local collection?
- Area definition storage: how are user-defined areas persisted?
- Waypoint proximity check: 100m geodesic or simple lat/lon distance?

6.4 Process

- When does convoy map cutover happen: end of V2.5 or separate task?
- Which features require V25_SPATIAL_ENABLED flag gating?
- How does recommit_docs handle the new stub files?

7. Complete Standing Rules (V2.5)

Original 21 rules from May 13 handoff plus all new rules from May 17:

11. Never auto-delete user data. All deletes require explicit confirmation.
12. Never embed API keys in APK. User manages their own keys.
13. Share is opt-in. Never upload without explicit user action.
14. Offline-first. Every feature works without connectivity.
15. Canvas independence. Map tiles are the user's choice.
16. Cumulative collection. Downloads add data. Only explicit removal deletes.
17. Routes never independently public. Only through ride invitations.
18. Private rides block sharing. Toggle suppresses share prompt entirely.
19. Published content always recoverable from collective (V3.0).
20. Trails and routes never removed individually. Source/region removal only.
21. First source ingested wins auto-resolve for duplicates.
22. Reusability: Ops files (logic) and Dialogs files (UI).
23. Every user data change is journaled. No exceptions.
24. Backup before every destructive operation. Automatic.
25. Journal stores before AND after state with geometry as WKT.
26. Schema must be hardened before application code.
27. Lifecycle triggers designed and tested before features built.
28. Patch scripts uniquely versioned Python files. Never reuse names.
29. Never commit `V3_FEATURES_ENABLED` or `V25_SPATIAL_ENABLED = true`.
30. Preserve full trail geometry precision.
31. Complete impact analysis before ANY code change.
32. All imports are tracks. Always. Convert to route is explicit type flip.
33. Track and route share same table with type field.
34. Tiles are never downloaded in isolation. Always follow spatial data.
35. Tile coverage maps to spatial data geometry, not full bbox.
36. Uploads are per-item on save via share prompt. Never area-based.
37. Uploads are spatial data only. Tiles never transit AWS.
38. V2.5 upload queue collects only. No processing.
39. Downloads use area for discovery, item selection for download.
40. Server browsing and downloads are paygated.
41. Local and server data never mixed on same display.
42. All operations user-initiated. No operations on launch. No auto-sync.
43. No MediaStore queries. Lesson from P10_T ANR.
44. Proximity check 100m for waypoint creation.
45. Long-press = waypoint creation. Cart HUD selection via button.
46. Phase 0 scaffolding ships first. All UI placeholders before feature code.
47. Share prompt controlled by `app_setting` and ride privacy toggle.
48. Two separate queues: tile queue and spatial data queue.

- 49. Area delete removes definition only. Data and tiles stay unless explicitly removed.
- 50. Never build V2.6 processing code during V2.5.
- 51. No upload button. Uploads happen through share prompt on save only.
- 52. Tile refreshes are never area-based. They happen when spatial data downloads.
- 53. Design all UI with HTML panel mockups before any coding.
- 54. One comprehensive patch per agreed design, not piecemeal changes.

END OF DECISION LOG | Review open issues before next coding session